UNIVERSIDAD DE VALLADOLID

GRADO EN INGENIERÍA BIOMÉDICA

TÉCNICAS COMPUTACIONALES EN BIOMEDICINA

Práctica 4. Optimización mediante algoritmos genéticos

CURSO 2024/2025

ÍNDICE

1. Objetivos y metodología 2

2. Temporización 3

3. Descripción general del problema bajo estudio 4

4. (SESIÓN 1) Diseño general de un procedimiento de optimización mediante algoritmos genéticos 4

4.1. Inspección de la base de datos 4

4.2. Solver ga en el Global Optimization toolbox de Matlab® 5

5. (SESIÓN 2) Población inicial y función objetivo 6

5.1. Población inicial 6

5.2. Implementación de la función objetivo o de ajuste (fitness) 7

6. (OPTATIVO) Funciones de representación 9

6.1. Representación de la evolución y resultado del algoritmo genético 10

7. Evaluación 10

8. Bibliografía 10

1. OBJETIVOS Y METODOLOGÍA

En esta última práctica del Bloque 1 se va a implementar un método de optimización de espacios de características mediante algoritmos genéticos. Para ello, se utilizará una base de datos clínicos de pacientes, con información procedente de diferentes fuentes de datos médicos: socio-demográficos, antropométricos, comorbilidades, test de calidad de vida o consumo de recursos, entre otras. Estas variables permitirán caracterizar a cada paciente y se emplearán como variables independientes de entrada a un modelo predictivo que habrá que optimizar. Los objetivos de la práctica consistirán en:

(i) Familiarizarse con el trabajo con datos clínicos reales.

(ii) Conocer e identificar las etapas que componen el proceso de optimización mediante algoritmos genéticos.

(iii) Comprender el funcionamiento y requisitos particulares de un algoritmo genético aplicado a la búsqueda de subespacios óptimos de variables como entrada a modelos predictivos automáticos.

(iv) Comprender la influencia de los diferentes parámetros que gobiernan la exploración del espacio de búsqueda en problemas de optimización basados en algoritmos genéticos. Forzar situaciones extremas que den lugar a una baja diversidad que limita el espacio de búsqueda y conduce al algoritmo genético a un mínimo local, para ser capaces de evitar dichas situaciones en problemas reales.

(v) Conocer las opciones y funcionalidades que ofrece el *Global Optimization toolbox* de Matlab para la resolución de problemas de optimización de forma general y, en particular, mediante algoritmos genéticos.

Cada alumno realizará la práctica individualmente. Se propone un desarrollo guiado y supervisado por el profesor, pero buscando que el alumno experimente, busque información de forma autónoma para resolver los problemas que le vayan surgiendo e interprete los resultados obtenidos de acuerdo a los conocimientos adquiridos en la parte teórica de la asignatura.

2. TEMPORIZACIÓN

Las sesiones prácticas se llevarán a cabo en el edificio en el **aula SIM-3247 del INDUVa (Escuela de Ingenierías Industriales, Sede Doctor Mergelina)**, de acuerdo a la siguiente planificación:

- Grupo **L1**
  - Sesión 1 (2 horas): **viernes 11** de octubre de **09:00 a 11:00**
  - Sesión 2 (2 horas): **viernes 18** de octubre de **09:00 a 11:00**
- Grupo **L2**
  - Sesión 1 (2 horas): **viernes 11** de octubre de **11:00 a 13:00**
  - Sesión 2 (2 horas): **viernes 18** de octubre de **11:00 a 13:00**

La distribución (orientativa) de las 4 horas de duración de la práctica será la siguiente:

Primera sesión de la práctica (2 horas)

1. Presentación (10')
2. Inspección de la base de datos (40')
3. Descanso (10')
4. *Global Optimization toolbox* y programación de algoritmos genéticos (40')
   a. Inspección del *solver* `ga` y sus funcionalidades (20')
   b. Diseño e implementación de la estructura del algoritmo genético (20')
5. Consideraciones finales (10')

Segunda sesión de la práctica (2 horas)

1. Presentación (10')
2. Población inicial y función objetivo (40')
   a. Especificación de la población inicial del algoritmo genético (10')
   b. Implementación de la función objetivo customizada (30')
3. Descanso (10')
4. Ejecución del algoritmo genético y análisis de resultados (10')
5. Implementación de funciones customizadas de representación (30') **(OPT)**
6. Consideraciones finales (10')

*** La evaluación de esta práctica (Práctica 4) se realizará conjuntamente con los contenidos de la Práctica 3, una vez finalizadas todas las sesiones de ambas prácticas. Consistirá en un cuestionario tipo test que se realizará al principio de la primera clase de teoría de la asignatura que se realice tras terminar las sesiones de laboratorio (típicamente la semana siguiente).**

3. DESCRIPCIÓN GENERAL DEL PROBLEMA BAJO ESTUDIO

En esta práctica vamos a construir un **modelo predictivo de probabilidad de reingreso hospitalario de pacientes respiratorios en los 30 días posteriores al alta**, es decir, identificar a aquellos pacientes con mayor probabilidad de que, tras ser dado de alta, vuelva a ser ingresado por la misma causa de nuevo en el hospital. Para ello, disponemos de una base de datos de **pacientes con enfermedad pulmonar obstructiva crónica (EPOC)**. La EPOC en una enfermedad respiratoria crónica caracterizada por una limitación al flujo aéreo persistente causada por una reacción inflamatoria exagerada de las vías aéreas y los pulmones frente a partículas o gases nocivos, principalmente el tabaco, aunque también puede ser por motivos laborales. La EPOC es la tercera causa de muerte a nivel mundial en adultos mayores de 40 años y es una de las patologías que causan mayor número de rehospitalizaciones (*i.e.*, reingresos). Esto supone un peor pronóstico para el paciente y un gran gasto de recursos sanitarios para el sistema de salud. Por ello, la creación de modelos predictivos de reingreso es un campo de estudio de gran interés en la actualidad.

Los pacientes de nuestra base de datos están caracterizados exhaustivamente mediante la recopilación de decenas de variables clínicas. Además, conocemos qué pacientes fueron dados de alta y reingresaron por el mismo motivo (exacerbación, *i.e.*, empeoramiento de su enfermedad). Utilizaremos esta información para crear un modelo predictivo de reingreso mediante un enfoque de entrenamiento supervisado (*i.e.*, basado en ejemplos conocidos). **Como clasificador utilizaremos un modelo de regresión logística** (RL), enfoque clásico muy empleado en biomedicina con el que ya habéis trabajado en las prácticas anteriores.

Disponemos de un número elevado de variables (variables independientes de entrada al modelo), pero de un número relativamente limitado de instancias/observaciones (*i.e.*, pacientes). Como sabemos, esto es un problema (*course of dimensionality*) que puede resultar en un modelo sobre-entrenado. Para minimizarlo, se ha decidido realizar una **reducción de dimensionalidad mediante una selección de variables aplicando algoritmos genéticos**. Básicamente, el algoritmo genético evalúa múltiples modelos de RL compuestos por diferentes variables de entrada y se queda con la combinación de variables que maximiza una función objetivo. Como sabemos, la búsqueda de ese subconjunto óptimo de variables se realiza tratando de imitar las leyes de la evolución natural. En esta práctica, **la función objetivo a optimizar será una métrica de rendimiento predictivo, la cual buscaremos maximizar**.

4. (SESIÓN 1) DISEÑO GENERAL DE UN PROCEDIMIENTO DE OPTIMIZACIÓN MEDIANTE ALGORITMOS GENÉTICOS

En esta primera sesión de la práctica el alumno se familiarizará con las variables clínicas que se emplearán para crear el modelo predictivo de reingreso, así como con las opciones y funcionalidades que ofrece la función que implementa algoritmos genéticos en Matlab®, el *solver* de optimización `ga`, del *Global Optimization toolbox*.

4.1. Inspección de la base de datos

En primer lugar, cargue la base de datos, que se encuentra disponible entre los recursos de la Práctica 4 en el apartado correspondiente del Campus Virtual de la asignatura ("Practica-4_TCB.mat"). A continuación, estudie sus componentes y la estructura de los mismos. Para ello, utilice el comando `load` o la funcionalidad *Import Data* de la barra de herramientas de Matlab®. Analice los diferentes tipos de variables que caracterizan a cada paciente desde un

punto de vista tanto clínico (grupos de variables socio-demográficas, antropométricas, comorbilidades, cuestionarios de calidad de vida, consumo de recursos, caracterización de la enfermedad, estado del paciente en el ingreso, etc.) como técnico (cuáles son cuantitativas y cuáles cualitativas; cuáles son continuas, binarias y categóricas; rangos de datos, etc.).

- ¿Cuántos pacientes (instancias/observaciones) componen la base de datos?
- ¿Cuántas variables caracterizan a cada uno de los pacientes?
- ¿En cuántos grupos/clases se divide la variable dependiente (objetivo/target)? ¿Cuántas instancias (pacientes) forman cada clase? ¿Qué representa en términos clínicos cada valor de la variable dependiente?

4.2. Solver ga en el Global Optimization toolbox de Matlab®

A continuación, utilice la ayuda de Matlab® (icono *Help* de la barra de herramientas) para buscar información sobre la implementación de algoritmos genéticos que realiza este entorno de programación. Podrá resultar de utilidad el Anexo I de esta Práctica 4, en el que se detallan y explican las funcionalidades más relevantes de entre todas las herramientas que ofrece Matlab para implementar un algoritmo genético dentro del *Optimization Toolbox*. Lea con detenimiento este anexo y revise los argumentos de entrada, de salida y las opciones de configuración del *solver* **`ga`**.

En esta práctica vamos a trabajar con los siguientes parámetros configurables del algoritmo genético:

- Recombinación: estrategia de cruce y porcentaje de individuos reemplazados.
- Mutación: estrategia de mutación y probabilidad de mutación.
- Población: tipo de codificación de los datos, tamaño de la población, especificación de la población inicial y elitismo.
- Función objetivo: implementación de la función de ajuste/*fitness* y función de escalado de los valores de ajuste.
- Selección: estrategia de selección de padres.
- Representación: intervalo de representación, funciones de representación,
- Criterios de parada: número máximo de generaciones, número máximo de generaciones con ganancia en la función objetivo menor que la tolerancia, tiempo máximo de ejecución y tiempo máximo de ejecución con ganancia en la función objetivo menor que la tolerancia.

Implemente una función de Matlab® en la que vaya diseñando la estructura de opciones (`optimoptions`) y ejecución (`ga`) del algoritmo genético. Como argumentos de entrada, la función de búsqueda del subconjunto óptimo de variables tendrá:

- La matriz de datos (*MD*) y el vector de targets (*Y*) de la base de datos de trabajo.
- Los parámetros configurables básicos de un algoritmo genético:
  - Tamaño de la población de soluciones potenciales (*PopulationSize*)
  - Población “semilla” inicial (*PInicial*)
  - Probabilidad de cruce (*Pc*, equivalente a porcentaje de reemplazo)
  - Probabilidad de mutación (*Pm*)
  - Número máximo de generaciones (*NGen*)

Como argumentos de salida, la función devolverá:

- El individuo óptimo, es decir, la secuencia de bits que identifica qué variables han sido seleccionadas y cuáles no.
- La dimensión del subespacio óptimo, es decir, el número de variables seleccionadas.
- El valor de ajuste/*fitness* óptimo.
- El valor de ajuste/*fitness* promedio de la población de la última generación.

Como punto de partida, utilice los valores y estrategias recomendados en clase para los parámetros configurables y el resto de opciones que se pueden fijar mediante la función de opciones de optimización `optimoptions`.

- ¿Cuál es la dimensión (número de variables de diseño, *nvars*) que debemos pasar al solver `ga` como segundo parámetro de entrada?
- ¿Qué tipo de datos para la población (*PopulationType*) debemos especificar en este problema particular de selección del subconjunto óptimo de variables?
- ¿Cuáles son los valores recomendados de probabilidad de cruce (porcentaje de reemplazo), probabilidad de mutación y número máximo de generaciones?
- ¿Qué estrategias de recombinación/cruce, mutación y selección de individuos ofrece el *solver* `ga` del *Global Optimization toolbox* de Matlab®?

5. (SESIÓN 2) POBLACIÓN INICIAL Y FUNCIÓN OBJETIVO

La segunda sesión de la práctica se centrará en dos etapas fundamentales en la evolución de un algoritmo genético: (i) la población inicial y (ii) la función objetivo. En la primera parte de la sesión, el alumno especificará una población inicial, que será el punto de partida (semilla) del algoritmo genético. Finalmente, el alumno implementará una función objetivo acorde con el problema bajo estudio (selección de variables de entrada a un modelo predictivo).

A continuación, se proporciona un paquete de funciones de Matlab® que pueden resultar de utilidad durante la programación de las funcionalidades solicitadas en la segunda sesión de la práctica:

```
cvpartition    ones      training   size     std    sum

glmfit    mean      zeros     glmval    test      repmat
```

El alumno será el responsable de investigar cuál es la funcionalidad de cada una de ellas a través de la ayuda de Matlab® y las utilizará cuando crea conveniente.

5.1. Población inicial

La población de partida debe garantizar que se va a explorar convenientemente todo el espacio de búsqueda. Dependiendo de la naturaleza del problema bajo estudio, no siempre es adecuado inicializar la población de partida de forma aleatoria. En esta práctica, el objetivo es seleccionar el subconjunto óptimo de variables de entrada a un modelo predictivo, donde cada **individuo (solución potencial, NO PACIENTE)** de la población **(espacio de búsqueda, NO POBLACIÓN DE ESTUDIO)** es un vector de 1's y 0's que representa un subconjunto particular de variables (1: la variable está incluida; 0: la variable no está incluida).

- ¿Qué podría pasar si inicializamos nuestro algoritmo genético con una población inicial aleatoria?

En esta práctica se trabajará con una población inicial prefijada, que se encuentra entre los recursos de la práctica puestos a disposición del alumno. Identifique la matriz que representa la población inicial de soluciones potenciales y **estudie cómo pasarla al algoritmo genético a través del parámetro/argumento adecuado** de la estructura de opciones que recibe el solver `ga` (la estructura que se crea con `optimoptions`; ver Anexo I, pp. 11-12).

- ¿Cuál es el número de filas y de columnas que tiene la matriz con la población de partida del algoritmo genético? ¿Por qué?
- ¿A través de qué argumento de entrada de `optimoptions` (par *Name*, *Value*) se especifica la matriz que representa el punto de partida del proceso de optimización?

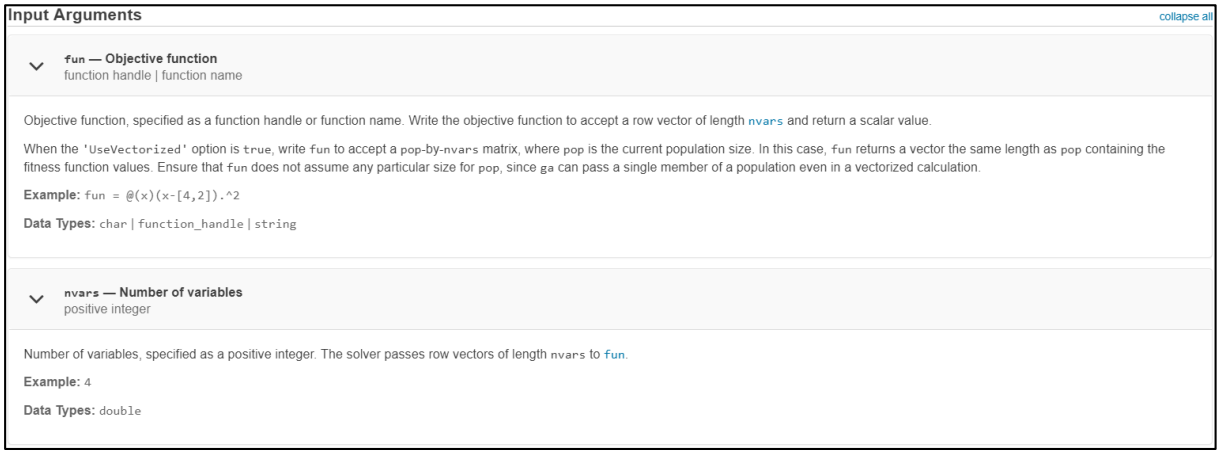
5.2. Implementación de la función objetivo o de ajuste (fitness)

Como enfoque de clasificación para diseñar el modelo predictivo de reingreso se va a emplear un modelo RL. En los problemas de optimización en los que intervienen modelos predictivos, es común fijar como función objetivo una métrica de rendimiento predictivo. Como nuestro problema bajo estudio es binario (pacientes que reingresan *vs.* pacientes que no reingresan), lo que estamos diseñando es básicamente un clasificador binario. En estos casos, **la métrica de rendimiento a optimizar más común es la precisión (Acc) del clasificador**:

$$Acc = \frac{Nº\ de\ pacientes\ correctamente\ clasificados}{Nº\ total\ de\ pacientes}$$

Por tanto, **nuestra función objetivo deberá customizarse** (será lo que en Matlab se denomina *custom function*) para poder recibir los datos necesarios para crear un modelo RL y calcular y devolver su Acc. **A la hora de crear <u>funciones que se van a llamar/ejecutar dentro del algoritmo genético</u>**, deberán tenerse en cuenta las siguientes características:

- La **función objetivo** del *solver* `ga` tiene una **estructura predefinida que hay que respetar** → Busque en la ayuda de Matlab cómo es la estructura básica de la función objetivo/*fitness*. A continuación, se proporciona una captura de pantalla que le resultará de utilidad.



Input Arguments collapse all

fun — Objective function
function handle | function name

Objective function, specified as a function handle or function name. Write the objective function to accept a row vector of length `nvars` and return a scalar value.

When the `'UseVectorized'` option is `true`, write `fun` to accept a `pop`-by-`nvars` matrix, where `pop` is the current population size. In this case, `fun` returns a vector the same length as `pop` containing the fitness function values. Ensure that `fun` does not assume any particular size for `pop`, since `ga` can pass a single member of a population even in a vectorized calculation.

Example: `fun = @(x)(x-[4,2]).^2`

Data Types: `char` | `function_handle` | `string`

nvars — Number of variables
positive integer

Number of variables, specified as a positive integer. The solver passes row vectors of length `nvars` to `fun`.

Example: `4`

Data Types: `double`

- **A mayores de los argumentos de entrada** que deben ser **fijos** de acuerdo a la estructura que debemos respectar, también **se le pueden pasar argumentos de entrada adicionales**. → Busque en la ayuda de Matlab cómo pasar argumentos adicionales (*Passing Extra Parameters* mediante una función anónima (*Anonymous Functions*)). A continuación, se proporciona una captura de pantalla que le resultará de utilidad.

ga R2020a

Find minimum of function using genetic algorithm

collapse all in page

Syntax

```
x = ga(fun,nvars)
x = ga(fun,nvars,A,b)
x = ga(fun,nvars,A,b,Aeq,beq)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub,nonlcon)
x = ga(fun,nvars,A,b,Aeq,beq,lb,ub,nonlcon,options)
x = ga(fun,nvars,A,b,[],[],lb,ub,nonlcon,IntCon)
x = ga(fun,nvars,A,b,[],[],lb,ub,nonlcon,IntCon,options)
x = ga(problem)
[x,fval] = ga(___)
[x,fval,exitflag,output] = ga(___)
[x,fval,exitflag,output,population,scores] = ga(___)
```

Description

example

`x = ga(fun,nvars)` finds a local unconstrained minimum, `x`, to the objective function, `fun`. `nvars` is the dimension (number of design variables) of `fun`.

Note

Passing Extra Parameters (Optimization Toolbox) explains how to pass extra parameters to the objective function and nonlinear constraint functions, if necessary.

Como en cualquier proceso de optimización de parámetros libres de un modelo predictivo, es necesario disponer de (i) un primer grupo de datos de entrenamiento para crear un modelo (los coeficientes del modelo RL en este caso) para los diferentes "valores" del "parámetro" que queremos configurar y (ii) un segundo grupo de validación independiente en el que calcular la métrica de rendimiento (Acc en este caso) para decidir cuál es el "valor" óptimo. Esto es lo que se denomina estrategia *hold-out*.

➔ En esta práctica, el parámetro que queremos optimizar es un poco especial: el subconjunto óptimo de variables de entrada al modelo RL.

Por tanto, nuestra función objetivo de cálculo de la Acc deberá recibir (además de los parámetros fijos que haya que respetar) dos matrices de datos de entrada (*Mtrain* y *Mtest*) con sus correspondientes targets (*Ytrain* e *Ytest*) para:

- **Obtener los coeficientes del modelo RL empleando el conjunto de entrenamiento**.
- **Calcular la Acc** del modelo creado, que será el valor de *fitness* de cada individuo, **en el conjunto de datos de test**.

Para completar adecuadamente esta etapa, deberá tener en cuenta que:

- Tendrá que **dividir aleatoriamente la base de datos** que se proporciona en la práctica en dos grupos, entrenamiento y test.
    - Esto se hace típicamente en una **proporción del 60% – 40%**.
    - Es recomendable que esta partición sea **aleatoria pero estratificada**, es decir, que las proporciones 60%-40% se mantengan en todas las clases involucradas en el problema: pacientes que reingresan (clase '1') y pacientes que no reingresan (clase '0').
    - Lógicamente, la misma división de índices que se aplica a los datos (MD) se debe aplicar también a los targets (Y).
- En el proceso de diseño del modelo, para evitar grandes diferencias en las escalas y rangos de datos de las variables que puedan dar lugar a inconsistencias en el modelo,

siempre se debe **trabajar con matrices de datos estandarizadas**, es decir, de **media nula y desviación estándar unidad**. Para ello deberá:

- Estandarizar los datos de entrenamiento con las medias y desviaciones estándar de la propia matriz de datos de entrenamiento.
- Estandarizar los datos de test empleando las medias y desviaciones estándar de la matriz de entrenamiento.

Finalmente, una vez implementada la función objetivo/*fitness* customizada dentro de un archivo *.m* independiente, **pasaremos al solver `ga` dicha función como argumento de entrada (primer argumento de entrada), mediante** lo que en Matlab se denomina **un *function handle***: no estamos ejecutando la función, sino que estamos pasando la "estructura" de la función como argumento de entrada para que sea llamada y ejecutada múltiples veces dentro del *solver* `ga`. En el ejemplo que se detalla en la explicación del paso de parámetros extra mediante una función anónima que acabáis de ver, es posible identificar también cómo pasar este tipo de funciones como argumento de entrada (*function handle*: @).

Una vez especificada la población inicial y completada la función objetivo **ya podrá realizar una primera ejecución del algoritmo genético** con los valores que ha seleccionado (o dejado a sus valores por defecto) para los parámetros configurables (tamaño de la población, estrategia y probabilidad de cruce (proporción de reemplazo), estrategia y probabilidad de mutación, elitismo y número máximo de generaciones).

- Al ejecutar la función de optimización, ¿es posible visualizar en tiempo real la evolución del algoritmo genético? ¿Qué debería hacer para visualizar el valor de ajuste del mejor individuo y el ajuste promedio de toda la población?
- ¿Ocurre algo inesperado al ejecutar y visualizar la evolución del algoritmo genético? ¿Cómo podría solucionarse?
- ¿Cuál es el individuo óptimo seleccionado tras la finalización de la última iteración/generación? ¿Qué variables "reales" representan esa cadena de bits?
- **OPTATIVO**. Varíe los valores de los parámetros de probabilidad de cruce, mutación y elitismo para incrementar/reducir la diversidad de la población. ¿Cómo cambian estos parámetros la exploración del espacio de búsqueda?

6. (OPTATIVO) FUNCIONES DE REPRESENTACIÓN

La última parte de la práctica, de carácter optativo, se centrará en las opciones de representación de los resultados. El alumno deberá crear dos funciones de representación customizadas, que se adaptarán a las características particulares del problema bajo estudio: selección del subconjunto óptimo de variables de entrada a un modelo predictivo.

A continuación, se proporciona un conjunto de funciones de Matlab® que pueden resultar de utilidad durante la programación de las funcionalidades solicitadas:

```
length        sum    plot      mean    hold   min
          axis      xlabel         ylabel
```

El alumno será el responsable de investigar cuál es la funcionalidad de cada una de ellas a través de la ayuda de Matlab® y las utilizará cuando crea conveniente.

6.1. Representación de la evolución y resultado del algoritmo genético

En la primera ejecución del algoritmo genético, se habrá empleado una de las opciones de representación de la evolución del proceso de optimización que proporciona el *solver* `ga`. Sin embargo, esta representación no acaba de ajustarse del todo a nuestro problema particular. Para ello, se propone crear dos funciones *custom* de representación de diferentes resultados intermedios del proceso de optimización.

Deberá crear: (i) una función para representar conjuntamente el valor de ajuste óptimo (Acc máxima **en términos porcentuales positivos**) del "mejor" individuo y el valor de ajuste promedio de toda la población para cada generación; y (ii) otra función independiente para representar el número de variables del subespacio óptimo para cada generación.

- Al igual que el resto de funciones que se pueden "pasar" a un algoritmo genético, las funciones de representación tienen una estructura básica que debe respetarse → Busque en la ayuda de Matlab cómo es la estructura básica de las funciones de representación.
- Además de los argumentos de entrada básicos que hay que respetar, también es posible pasar argumentos de entrada adicionales → Busque en la ayuda de Matlab cómo pasar argumentos adicionales (*Passing Extra Parameters* mediante una función anónima).

Finalmente, integre el paso de ambas funciones *custom* de representación en la estructura de su función de optimización del espacio de variables mediante algoritmos genéticos, manteniendo también la función propia '*gaplotbestf*' para visualizar sus similitudes y diferencias.

7. EVALUACIÓN

La evaluación de las prácticas 3 y 4 se realizará conjuntamente el **miércoles día 23 de octubre de 2024** al principio de la clase de teoría. Consistirá en un **cuestionario tipo test** sobre diversos aspectos vistos en los ejercicios propuestos en ambas prácticas y tendrá una **duración de 15 minutos**.

8. BIBLIOGRAFÍA

Ian H. Witten, Eibe Frank, Mark A. Hall, Data Mining. Practical Machine Learning Tools and Techniques, 3rd ed., Elsevier, 2011.

Oliver Kramer, Genetic Algorithm Essentials, Springer, 2017.

Daniel Álvarez, Roberto Hornero, J. Víctor Marcos, Félix del Campo. Feature selection from nocturnal oximetry using genetic algorithms to assist in obstructive sleep apnoea diagnosis. *Medical Engineering & Physics*, vol. 34, pp. 1049-1057, 2012.